

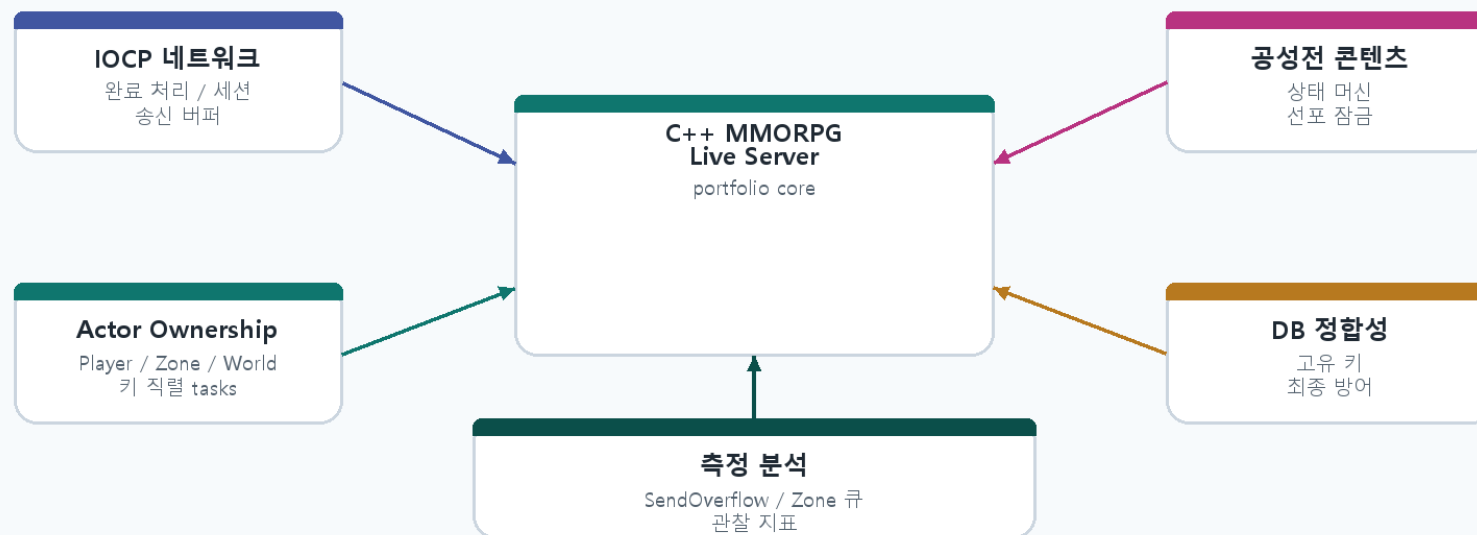
C++ MMORPG 라이브 서버 포트폴리오

IOCP / Actor 소유권 / 공성 상태 머신 / DB 경계 / 병목 분석

라이브 MMORPG 서버에서 반복적으로 마주치는 동시성, 상태 전이, DB 정합성, 관찰성 문제를 C++ 서버 프로젝트로 재구성한 포트폴리오입니다.

라이브 서버 포트폴리오 구성

프로젝트가 보여주는 핵심 역량



핵심: 상태 소유권, DB 경계, PvP 생명주기, 병목 분석

이광웅	연락처	저장소
C++ MMORPG 게임 서버 개발자	lgu4821@naver.com 010-5852-9537	github.com/KwangWoongLee/Portfolio

2. 경력과 포트폴리오 연결

실무 경험을 그대로 복제하지 않고, 같은 문제 유형을 개인 서버 프로젝트 안에서 설명 가능한 구조로 바꿨습니다.

경력 연결

- 대규모 PvP 콘텐츠 경험은 공성 상태 머신과 선포 잠금으로 연결
- 상품/보상/수령 경험은 DB 정합성과 중복 방지로 연결
- Grafana/로그/DB 분석 경험은 SendOverflow, Zone 큐 병목 분석으로 연결
- 운영툴/QA 협업 경험은 데모 로그와 실패 정책 설명으로 연결

이 프로젝트는 실제 라이브 서버 문제를 작은 코드 구조와 측정 가능한 흐름으로 재구성한 결과입니다.

경력과 포트폴리오 연결

포트폴리오 각 영역을 라이브 서버 경험과 연결



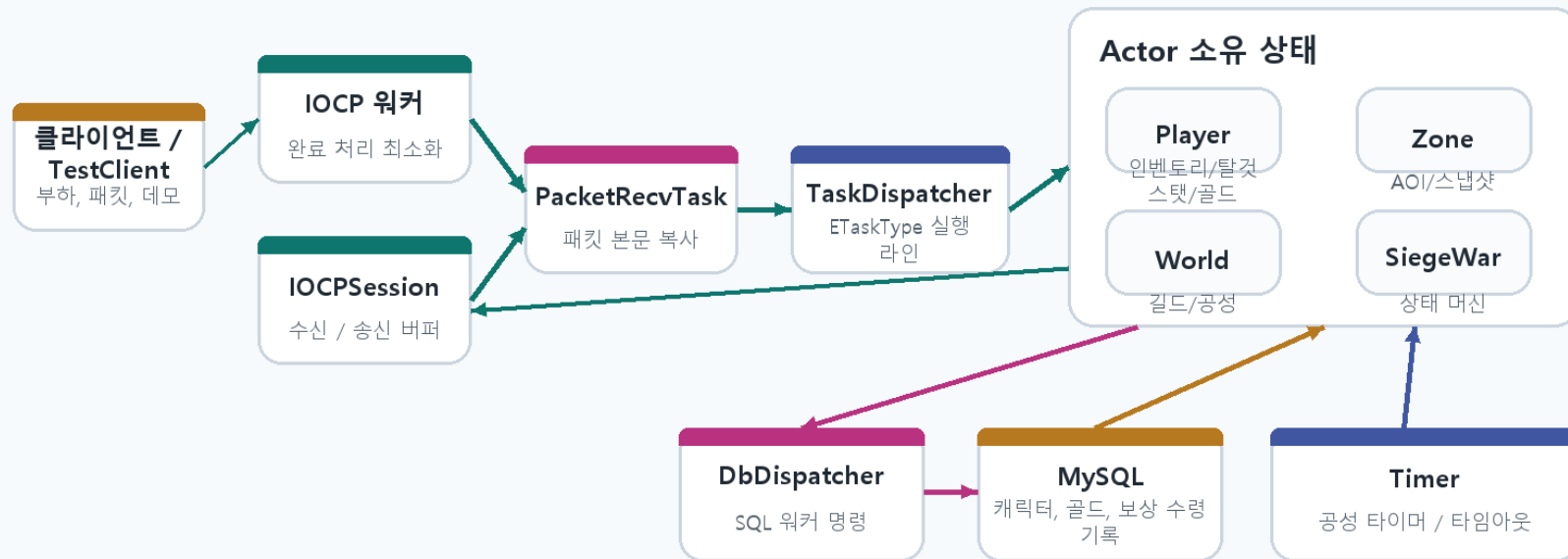
목적: 라이브 서비스 경험을 구체적인 서버 설계 근거로 연결

3. 전체 아키텍처

IOCP 워커는 완료 처리를 짧게 끝내고, 게임 상태 변경은 Actor 키 기준 직렬 작업으로 이동시킵니다.

포트폴리오 서버 아키텍처

IOCP 완료 처리, 키 직렬 Actor, DB 완료, 타이머 호출



원칙: IO/DB 워커는 짧게 처리하고, 게임 상태 변경은 소유 Actor로 되돌린다.

구간	책임
NetworkIO	IOCPSession, 수신/송신 버퍼, 종료 같은 네트워크 소유권 처리
TaskDispatcher	ETaskType과 키 기준으로 직렬 Actor 작업 분배
Actor 실행 라인	Player, Zone, World, SiegeWar 상태 변경 소유
DB/Timer	SQL 실행 후 완료 메시지 전달, 공성 타이머/타임아웃 처리

4. Actor 모델 선택 이유 - 소유권 경계

라이브 서비스에서 얻은 문제의식은 패킷, 틱, DB 완료 경로가 Player 내부 모델을 각자 수정할 때 경쟁 상태와 락 순서 위험이 커진다는 점이었습니다. 이 프로젝트에서는 외부 경로가 상태를 직접 바꾸지 않고 PlayerActor 키 직렬 실행 라인에 메시지를 모으도록 재구성했습니다.

설계 배경	포트폴리오 구현에서의 정리	PlayerActor 소유권 경계
경쟁 상태	Inventory, Mount, Stat 같은 Player 내부 모델의 동시 수정 위험	라이브 서비스의 경쟁 상태/데드락 문제의식을 명시적인 상태 소유권으로 재구성 문제 패턴 패킷 경로 아이템 사용 / 장착 모델 변경 틱 경로 회복 / 탈것 스탯 갱신 Player 내부 모델 Inventory / Mount / Stat 공유 변경 가능 상태 소유자 불명확 락 선택의 대가 모델별 락 -> 락 순서 / 데드락 위험 Player mutex -> 넓은 결합 구간 포트폴리오 설계 NetworkIO 패킷 메시지 Timer 틱 메시지 DB 완료 결과 메시지 PlayerActor#1001 단일 직렬 실행 라인 Packet:Use -> Tick:Regen -> DB:SaveAck Inventory / Mount / Stat / Gold 변경 짧은 Actor 작업 + 큐 지연 측정 결과: IO, 타이머, DB 경로는 메시지를 만들고 PlayerActor가 변경 순서와 큐 압력을 소유한다.
모델별 락	모델별 락은 호출 순서가 복잡해져 데드락 가능성 증가	
Player 전체 락	단순하지만 이동/전투/인벤토리/저장 요청이 한 락에 결합	
PlayerActor	NetworkIO, Timer, DB 완료를 메시징화하고 PlayerId 키 직렬 실행 라인에서 처리	
관찰 지표	Actor 작업을 짧게 유지하고 큐 지연 시간으로 병목 여부 확인	

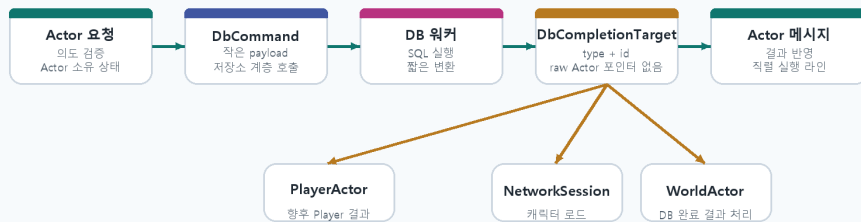
결과적으로 Player 내부 변경 가능 상태의 소유자와 실행 순서를 명확히 하고, 외부 워커는 상태 변경 대신 메시지 생성 경계에 머물도록 설계했습니다.

5. DB 완료 처리와 영속성 경계

DB 워커는 게임 상태를 직접 바꾸지 않고, DB 트랜잭션/SP와 완료 대상으로 정합성 경계를 나눕니다.

DB 완료 Boundary

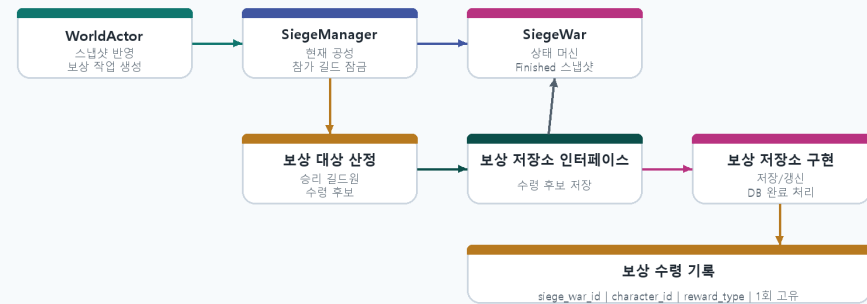
DB 워커는 SQL을 실행한 뒤 완료 결과를 소유 Actor로 전달한다.



이유: DB 워커가 Player/World/SiegeWar 상태를 직접 바꾸지 않도록 한다.

공성 보상 클래스 관계

전체 UML이 아니라 책임 경계에 초점을 둔 클래스 관계



핵심: 소유권, 보상 계획, 저장소 계층 경계, DB 최종 방어

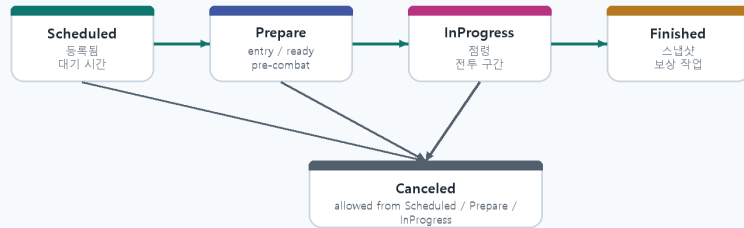
설계 포인트	설명
DB 완료 대상	SQL 결과를 NetworkSession, WorldActor 같은 처리 주체로 전달
DB 트랜잭션 / SP	골드/스탯처럼 DB 원자성이 필요한 변경은 저장소 계층 트랜잭션 또는 SP로 처리
DB 실행 이후 실패	재시도, 재조회, 연결 종료, 운영 로그, 먹등 키 정책으로 처리
보상 정합성	DB 고유 키로 캐릭터 단위 중복 지급 방어

6. 공성전 생명주기와 선포 잠금

공성전은 장시간 PvP 상태 머신으로 설명하고, 선포 잠금/결제/환불은 그 안의 실패 경로 사례로 넣습니다.

SiegeWar State Machine

장시간 PvP 콘텐츠를 명시적 전이와 타이머 호출로 관리한다.



WorldActor는 스냅샷을 반영하고 타이머를 재예약한 뒤 Finished 이후 보상 작업을 만든다.

공성 선포 잠금

길드 잠금은 전투 시작이 아니라 유효한 선포 예약 시점부터 시작된다.



실패 경로 우선

결제 timeout, 결제 실패, 취소, 공성 종료는 모두 잠긴 길드 id를 해제해야 한다.

- Scheduled -> Prepare -> InProgress -> Finished 상태 전이를 타이머 호출로 진행
- Canceled는 운영 취소와 예외 경로를 별도 상태로 고정
- 길드 해체 방지 의도라면 잠금은 InProgress가 아니라 선포 예약 성공 시점부터 적용
- Finished 스냅샷 이후 보상 대상을 산정하고 DB 고유 키로 중복을 방어

7. 측정 분석 - SendOverflow

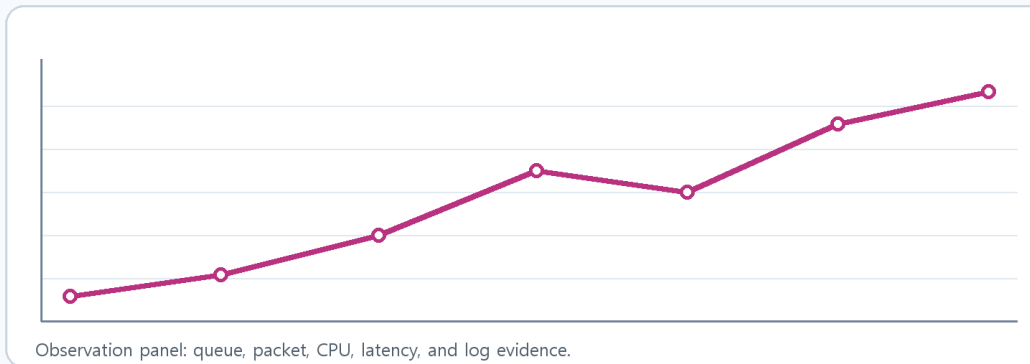
SendOverflow는 서버 송신 코드만 보지 않고, 클라이언트 수신 처리와 TCP 역압까지 함께 봐야 합니다.

분석 지표

- 세션별 대기 송신 바이트
- 봇 규모별 송신 overflow 횟수
- 클라이언트 수신 처리량
- 서버/클라이언트 CPU, 메모리, 네트워크 처리량
- 클라이언트 프로세스 분산 전후 비교

핵심 메시지는 서버 송신 로직만 보는 것이 아니라, 클라이언트 처리량과 TCP 역압을 함께 분리해 확인했다는 점입니다.

pending send bytes, overflow count, client receive throughput



8. 측정 분석 - 단일 Zone 큐

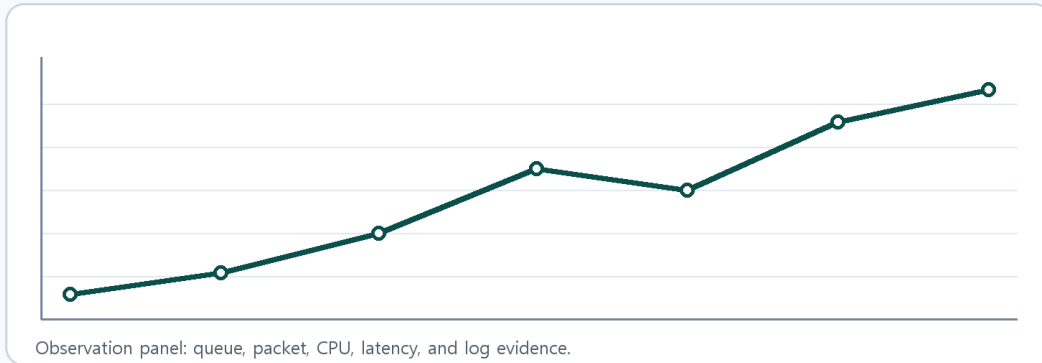
CPU가 낮아도 단일 Zone Actor 큐가 폭증하면 병목은 전체 서버가 아니라 특정 상태 소유 Actor에 있을 수 있습니다.

분석 지표

- 봇 규모별 Zone 큐 길이
- Actor 작업 지연
- 브로드캐스트 수 / 관찰자 수
- 서버 CPU가 낮는데 큐가 쌓이는 구간

개선 후보	의미
Zone 분할	단일 Actor 집중 완화
틱 배치	작업 비용 감소
브로드캐스트 감소	불필요한 송신 감소

zone queue length, task latency, observer and broadcast pressure



9. 구현 범위와 검증

구현 범위와 GoogleTest 검증 범위를 함께 정리해, 어떤 규칙을 코드로 확인했는지 보여줍니다.

구현/검증 영역	내용	검증 포인트	내용
네트워크	IOCP 세션, 패킷 작업, 송신 버퍼	StateMachineTests	전이 hook과 guard 거부, 허용되지 않은 전이 확인
Actor	Player/Zone/World/SiegeWar 키 직렬 실행	SiegeWarTests	점령 유예, 최종 마감, 수비 길드 갱신 규칙 확인
공성전	상태 머신, 선포 잠금, 결제/환불	RewardRulesTests	보상 후보 생성, 무승부 건너뛰기, 중복 작업 방어 확인
DB	DbCompletionTarget, 저장소 계층 트랜잭션, 보상 정합성	향후 보완	DB 실패 정책 확장, 워커 설정, 틱 배치, Zone 분할
측정	SendOverflow, Zone 큐 분석 관점		
GoogleTest	공성전 상태 전이와 핵심 규칙 검증		

10. 마무리 - 이 포트폴리오가 보여주는 것

C++ 구현 자체보다, 라이브 서버에서 상태 소유권과 정합성, 관찰성, 병목 판단을 어떻게 설계했는지를 보여줍니다.

- 경쟁 상태/데드락 경험을 PlayerActor 단위 상태 소유권과 키 직렬 실행으로 정리함
- DB 완료 대상과 저장소 계층 트랜잭션으로 메모리/DB 경계를 설명함
- GoogleTest로 공성전 상태 전이와 핵심 규칙을 검증함
- 공성전 상태 머신으로 장시간 PvP 생명주기와 실패 경로를 관리함
- SendOverflow와 Zone 큐 사례로 병목 판단 과정을 설명함

구현 구조, 검증 범위, 측정 관점을 함께 제시해 라이브 MMORPG 서버 개발자로서의 문제 해결 방식을 보여줍니다.

이 포트폴리오가 보여주는 것

구현은 라이브 서비스 문제의식과 연결된다.

